PROGRAMMATION ORIENTéE OBJET (POO)

1. Définition :

La **programmation orientée objet (POO)** est un **paradigme de programmation** qui organise le code autour de **classes et d'objets**, plutôt que des fonctions et des variables seules.

- **Classe** : modèle ou plan qui définit les caractéristiques et comportements d'un objet.
- **Objet** : instance d'une classe, un exemplaire concret avec ses propres valeurs.

2. Concepts clés de la POO :

Concept	Définition
Classe	Modèle pour créer des objets, contient attributs et méthodes
Objet	Instance de classe, peut avoir ses propres valeurs
Attribut/Propriété	Variable appartenant à un objet
Méthode	Fonction définie dans une classe, qui agit sur ses objets
Encapsulation	Protéger les données via private et public
Héritage	Une classe peut hériter des propriétés et méthodes d'une autre classe
Polymorphisme	Une même méthode peut se comporter différemment selon l'objet
Constructeur	Méthode spéciale pour initialiser un objet

3. Exemple 1 : Définir une classe et créer un objet

using System;

```
class Personne{
   // Attributs (propriétés)
   public string Nom;
   public int Age;
   // Méthode
   public void SePresenter(){
      Console.WriteLine("Je m'appelle " + Nom + " et j'ai " + Age + " ans.");
   }
}
class Program{
   static void Main(){
      // Création d’un objet
      Personne p1 = new Personne();
      p1.Nom = "Alice";
      p1.Age = 25;
      p1.SePresenter();
   }
}
```

Explication :

- class Personne définit le modèle
- Personne p1 = new Personne();// crée un objet
- p1.SePresenter(); //appelle la méthode de l’objet

4. Exemple 2 : Constructeur

- **Définition :**

  Un **constructeur** est une méthode spéciale utilisée pour initialiser un objet lors de sa création.

```
class Personne{
   public string Nom;
   public int Age;
   // Constructeur
   public Personne(string nom, int age){
      Nom = nom;
      Age = age;
   }
   public void SePresenter(){
      Console.WriteLine("Je m'appelle " + Nom + " et j'ai " + Age + " ans.");
   }
}
class Program{
   static void Main(){
      Personne p2 = new Personne("Bob", 30);
      p2.SePresenter();
```

```
    }
}
```

Explication :

- Personne(string nom, int age) → initialise les propriétés directement lors de la création.

5. Exemple 3 : Encapsulation (propriétés privées)

```
class CompteBancaire{
    private double solde; // Encapsulation : solde privé
    public void Deposer(double montant){
        solde += montant;
    }
    public void AfficherSolde(){
        Console.WriteLine("Solde : " + solde);
    }
}
class Program{
    static void Main(){
        CompteBancaire compte = new CompteBancaire();
        compte.Deposer(500);
        compte.AfficherSolde();
    }
```

```
}
```

Explication :

- solde est private → ne peut pas être modifié directement depuis l'extérieur .
- Les méthodes Deposer et AfficherSolde permettent d'accéder en toute sécurité .

6. Exemple 4 : Héritage

```
class Animal{
    public string Nom;
    public void Manger(){
        Console.WriteLine(Nom + " mange.");
    }
}
// Chat hérite de Animal
class Chat : Animal{
    public void Miauler(){
        Console.WriteLine(Nom + " miaule.");
    }
}
class Program{
    static void Main(){
        Chat chat = new Chat();
```

```
        chat.Nom = "Minou";
        chat.Manger(); // méthode héritée
        chat.Miauler(); // méthode spécifique
    }
}
```

Explication :

- Chat : Animal → Chat hérite des propriétés et méthodes de Animal.
- Héritage permet de **réutiliser et étendre le code.**

7. Exemple 5 : Polymorphisme (méthodes surchargées)

```
class Calcul{
    public int Addition(int a, int b){
        return a + b;
    }
    public double Addition(double a, double b){
        return a + b;
    }
}
class Program{
    static void Main(){
        Calcul c = new Calcul();
        Console.WriteLine(c.Addition(2, 3));      // int
```

```
        Console.WriteLine(c.Addition(2.5, 3.7));   // double
    }
}
```

Explication :

- Même nom de méthode, mais différents types de paramètres → **surcharge.**
- Permet d'utiliser une méthode de manière flexible selon le type de données.